Project-Low LevelDesign

on

IaC Provisioning for Real Estate System

Course Name: DevOps

Institution Name:Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Kushagra Gupta	EN22CS301546
2	Khushi Paliwal	EN22CS301505
3	Kritika Nagar	EN22CS301526
4	Kishna Sharma	EN22CS301510
5	Jayesh Jain	EN22CS301465
6	Kartik Gupta	EN22CS301488

Group Name:Group 07D5

Project Number:DO-07

Industry Mentor Name: Mr.Vaibhav

University Mentor Name:Prof.Ritesh Joshi

Academic Year:2025–2026

TableofContents

1. Introduction.
   1.1. Scopeofthedocument.
   1.2. IntendedAudience
   1.3. Systemoverview.

2. SystemDesign.
   2.1. ApplicationDesign
   2.2. ProcessFlow.
   2.3. InformationFlow.
   2.4. ComponentsDesign
   2.5. KeyDesignConsiderations
   2.6. APICatalogue.

3. References

1. Introduction

1.1 Scope of the Document

This document describes the **low-level design and architecture of the IaC Provisioning for Real Estate System**. The purpose of the system is to automate the provisioning and deployment of infrastructure required for a real estate application using **Infrastructure as Code (IaC)** tools.

Modern applications require scalable and reliable infrastructure to support development and deployment processes. Manual infrastructure setup is time-consuming, error-prone, and difficult to maintain. The IaC Provisioning system solves this problem by automating infrastructure creation, configuration, and deployment using tools such as **Terraform or Ansible**.

This system provisions infrastructure resources such as **cloud instances, container environments, and networking components** automatically. It also integrates **CI/CD pipelines** to build, test, and deploy applications using container technologies..

1.2 Intended Audience

- DevOps Engineers
- Developers
- System Administrators
- Evaluators

1.3 System Overview

The **IaC Provisioning for Real Estate System** is a DevOps-based infrastructure automation platform that provisions and manages infrastructure required for a real estate application.

The system uses **Infrastructure as Code tools such as Terraform or Ansible** to automatically create and configure infrastructure resources such as **cloud instances, networking components, and container environments**.

The application code is stored in a **Git repository**, which triggers an automated **CI/CD pipeline using Jenkins or GitHub Actions** whenever new code is pushed.

The pipeline builds a **Docker container image** of the application and pushes it to a container registry. The application is then deployed to a **Kubernetes cluster**, which manages container orchestration, scaling, and availability.

This automated system enables faster deployment, consistent infrastructure management, and improved scalability for the real estate platform.

2. System Design

2.1 Application Design

The **IaC Provisioning for Real Estate System** follows a three-layer architecture consisting of the **frontend, backend, and infrastructure layer**.

The **frontend** provides the user interface where users can interact with the real estate platform to view property listings, register, and perform other actions. The frontend can be developed using modern web technologies such as **React**, which allows users to access the application through a web browser.

The **backend** is responsible for handling user requests, processing application logic, and managing communication with the database. It can be developed using **Node.js and Express**, which provide REST APIs for operations such as user authentication, property management, and data processing. The **database** (such as MongoDB or MySQL) stores information related to users, properties, and transactions.

The infrastructure required to run the application is provisioned using **Infrastructure as Code tools such as Terraform or Ansible**, while the application is containerized using **Docker** and deployed on **Kubernetes** to ensure scalability and efficient resource management.

The **CI/CD pipeline layer** is implemented using **Jenkins or GitHub Actions**. This pipeline automatically builds, tests, and deploys the application whenever changes are pushed to the repository. The pipeline ensures continuous integration and continuous delivery of the application.

The overall application workflow is as follows:

1. A developer pushes the application code and infrastructure scripts to the **Git repository**.
2. The **CI/CD pipeline** is automatically triggered.
3. The pipeline builds the application and creates a **Docker container image**.
4. The Docker image is pushed to a **container registry**.
5. Infrastructure resources are provisioned using **Terraform or Ansible**.
6. The application container is deployed to the **Kubernetes cluster**.
7. Kubernetes manages the running containers and ensures scalability and reliability of the real estate application.

2.2 Process Flow

Developer pushes code to GitHub

GitHub Actions triggers CI/CD pipeline

Docker image is built automatically

Infrastructure is provisioned using Terraform

EC2 instance is created/configured

Docker container is deployed on EC2

Application becomes accessible via public IP

2.3 Information Flow

Developer → GitHub → CI/CD Pipeline → Docker Image → AWS EC2 → User Browser

- Input: Source code updates
- Processing: Build + deploy automation
- Output: Live application

2.4 Components Design

1. GitHub (Version Control)

- Stores codebase
- Maintains Dockerfile and configs

2. Docker

- Builds application image
- Ensures consistent runtime environment

3. AWS EC2

- Virtual machine hosting the application
- Runs Docker container

4. Terraform (IaC)

- Automates infrastructure creation
- Defines EC2 instance in code

5. CI/CD (GitHub Actions)

- Automates:
  - Build process
  - Deployment process

6. Application (Real Estate System)

- User autentication system
- Property listing platform

2.5 Key Design Considerations

• **Automation:** Infrastructure provisioning and deployment are automated using **Terraform and CI/CD pipelines**, reducing manual configuration.

• **Scalability:** The system can scale to support more users and property listings by deploying additional instances if required.

• **Portability: Docker containers** ensure the application runs consistently across development and production environments.

• **Reliability:** Automated build and deployment processes help reduce errors and improve system.

• **Security:** Secure access to cloud resources is maintained using **SSH keys and authentication mechanisms**.

• **Maintainability:** Infrastructure and application configurations are managed using code, making updates easier and more manageable.

2.6 API Catalogue

API	Method	Description
/	GET	Loads the real estate application
/register	POST	Allows users to create an account
/login	GET	Autenticates users and allow secure login
/properties	GET	Retrieves available property listings

3. References

1. AWS EC2 Documentation
2. Docker Documentation
3. Terraform Documentation
4. GitHub Actions Documentation

4. Conclusions

This project demonstrates the implementation of DevOps practices for automating infrastructure provisioning and application deployment. It ensures a scalable, reliable, and reproducible system using modern tools like Docker, AWS, and Terraform.